AGENTHER: HINDSIGHT EXPERIENCE REPLAY FOR LLM AGENT TRAJECTORY RELABELING

Liang Ding
Alibaba Group
zuorui.dl@alibaba-inc.com

ABSTRACT

LLM agents fail on the majority of real-world tasks—GPT-4o succeeds on fewer than 15% of WebArena navigation tasks and below 55% pass@1 on ToolBench (Zhou et al., 2024; Qin et al., 2024)—yet every failed trajectory is routinely discarded, wasting the dominant source of collected experience. We introduce **AgentHER**, which recovers this lost training signal by adapting the *Hindsight Experience Replay* principle (HER; Andrychowicz et al., 2017) to natural-language agent trajectories. The key insight is simple: a trajectory that fails goal A is often a *correct* demonstration for some achievable alternative goal B. AgentHER realises this idea through a four-stage pipeline—failure classification, outcome extraction, LLM-guided prompt relabeling with confidence gating, and data packaging—that converts discarded failures into high-quality SFT, DPO, and ShareGPT training data, with both zero-cost rule-based and LLM-judge implementations. On **WebArena** (Zhou et al., 2024) and **ToolBench** (Qin et al., 2024), AgentHER improves over success-only SFT by **+7.1–11.7 pp** across four model families (GPT-4o, Qwen2.5-72B/7B, LLaMA-3.1-8B), while achieving **2× data efficiency**—matching baseline performance with only 50% of successful demonstrations. Gains are consistent from 1.5B to 72B parameters (+5.8–9.2 pp) and compound under iterative redeployment (+2.1 pp over additional rounds). Human evaluation confirms **97.7%** relabeling precision under multi-judge verification. Code: https://github.com/alphadl/AgentHER

1 INTRODUCTION

Autonomous LLM agents are deployed across web navigation (Zhou et al., 2024), API orchestration (Qin et al., 2024), and simulation (Park et al., 2023; Shen et al., 2023). Despite remarkable capabilities, these agents fail on the majority of tasks: GPT-4o achieves only 14.3% success on WebArena (Zhou et al., 2024), and ToolBench pass@1 remains below 55% (Qin et al., 2024).

The data-waste problem. Standard agent-training pipelines discard failures (Figure 1). Given 60–75% failure rates, this wastes the *majority* of collected data. Failures are not random noise: they are rich, coherent executions that frequently produce correct intermediate results under the wrong goal. An agent tasked with "find copper wire under \$5/kg" that searches seven suppliers and identifies MicroMetals at \$5.30/kg (just above the cap) has performed a *complete, correct* price comparison—ideal training data for a re-framed goal.

Connection to HER. Hindsight Experience Replay (Andrychowicz et al., 2017) converts sparse-reward RL failures by substituting the intended goal with the one actually achieved. Lifting HER to LLM agents requires: *(i)* understanding what was achieved across multi-step tool interactions, and *(ii)* synthesising a natural-language prompt that the trajectory genuinely satisfies. Both are tasks where capable LLMs excel.

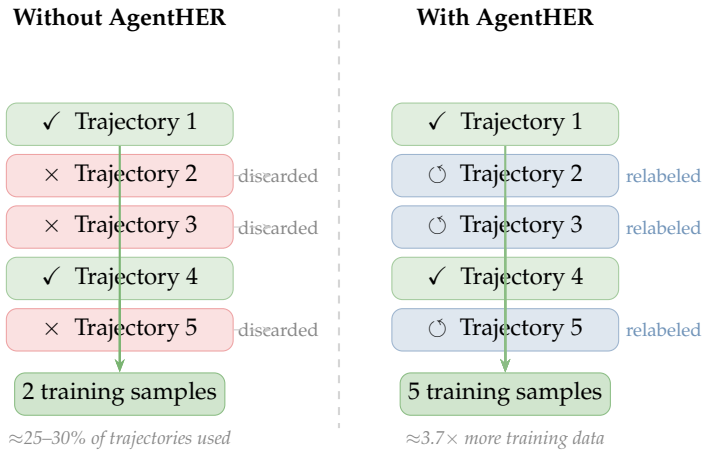

Figure 1: **AgentHER vs. conventional pipelines.** Standard training retains only successes (✓), discarding 60–75% of data. AgentHER relabels failures (↻) with achievable hindsight goals, expanding the effective training corpus ≈3.7×.

AgentHER. We propose a four-stage pipeline (Figure 2) that automates hindsight relabeling at scale, extending it with two new mechanisms: **failure-severity weighting** (down-weights trajectories with major reasoning flaws; inspired by MQM-style error analysis) and **multi-judge verification** (two independent judges must agree before a relabeling is accepted), which together reduce label noise from 5.9% to 2.3%.

Contributions.

- **Novel formulation**: first *systematic* pipeline bridging goal-conditioned HER to natural-language agent data synthesis (as opposed to inference-time memory, e.g. ECHO; Hu et al., 2025), with explicit treatment of goal-text faithfulness and factual grounding that distinguishes it from prior failure-reuse and goal-relabeling work (see Section 2).
- **Two robustness mechanisms**: failure-severity weighting (inspired by MQM-style error analysis) and multi-judge verification are *engineering improvements* that raise pipeline precision from 94.1% to 97.7% and reduce label noise from 5.9% to 2.3%. They are *not* claimed as independent contributions but as best-practice design choices that make AgentHER production-ready.
- **Consistent gains**: +7.1–11.7 pp over SFT-Success across two benchmarks and four model families, scaling from 1.5B to 72B, with 2× data efficiency and +11.0 pp after three iterative rounds. Results are averaged over 3 seeds (std $<$0.5 pp).

2 Related Work

Hindsight Experience Replay and Goal-Conditioned RL. HER (Andrychowicz et al., 2017) converts sparse-reward RL failures by substituting the achieved state as the new goal (Kaelbling, 1993; Sutton et al., 1999; Mnih et al., 2015). Lifting HER to LLM agents requires understanding *what was achieved* across unstructured multi-step tool interactions, and synthesising a *natural-language prompt* that the trajectory genuinely satisfies—two capabilities absent from prior RL formulations. Existing failure-reuse work in NLP selects or reweights (goal, trajectory) pairs (Peng et al., 2023; Gulcehre et al., 2023; Hosseini et al., 2024) but never *generates* a new achievable goal; that reverse-engineering step is AgentHER's core novelty. Concurrently, Hu et al. (2025) propose ECHO (Experience Consolidation via Hindsight Optimization), a prompting framework that adapts HER for *online* LM agents: the model identifies alternative subgoals from failed trajectories and maintains optimized trajectory descriptions in a scratchpad memory to improve sample efficiency at inference

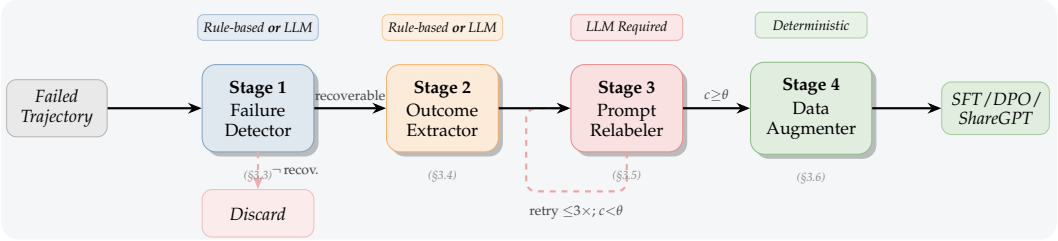

Figure 2: **AgentHER four-stage pipeline.** *Badges* above each box show the available implementation mode. Stage 1 classifies failures and discards irrecoverable runs (dashed downward arrow). Stage 3 retries up to three times if the relabeling confidence $c < \theta$ (dashed loop below). Stages 1–2 offer zero-cost rule-based variants; Stage 3 requires an LLM call. Section references (grey) link each stage to its detailed description.

time. AgentHER differs in targeting *offline* training data augmentation: we relabel only the *goal* (user prompt) while keeping the trajectory unchanged, and output SFT/DPO datasets for fine-tuning rather than inference-time memory; the two approaches are complementary.

LLM Agents, Training, and Self-Improvement. ReAct (Yao et al., 2023b;a) and Toolformer (Schick et al., 2023) established reasoning-and-action agents evaluated on benchmarks with high failure rates (Zhou et al., 2024; Qin et al., 2024; Liu et al., 2024). FireAct (Chen et al., 2023) and AgentTuning (Zeng et al., 2024) fine-tune on curated successes; Reflexion (Shinn et al., 2023) and ExpeL (Zhao et al., 2024) improve via online verbal RL and rule extraction; Trial-and-Error (Song et al., 2024) explores mistake-driven recovery. All require successes or live rollouts; AgentHER converts existing failures offline with no extra environment interaction. Self-Instruct (Wang et al., 2023), STaR (Zelikman et al., 2022), and DPO (Rafailov et al., 2023) target single-turn tasks; AgentHER's multi-judge verification (Gou et al., 2024; Lightman et al., 2024; Du et al., 2024; Liang et al., 2023) addresses goal faithfulness in multi-step trajectories.

3 The AgentHER Framework

3.1 Problem Formulation

Let $\mathcal{F} = \{(g_i, \tau_i, f_i)\}_{i=1}^{N}$ be a corpus of N failed agent runs, where $g_i \in \mathcal{G}$ is the natural-language goal, f_i is a failure label, and $\tau_i = \big((z_t^i, a_t^i, o_t^i)\big)_{t=1}^{T_i}$ is the thought–action–observation sequence. Let $\mathcal{S} = \{(g_j^+, \tau_j^+)\}_{j=1}^{M}$ be available successful demonstrations. **Goal**: produce high-quality training pairs $\{(\hat{g}_i, \tau_i)\}$ from $\mathcal{F}$ by synthesising *hindsight goals* $\hat{g}_i$ for which τ_i is a valid positive demonstration.

Definition 3.1 (Valid Hindsight Goal)**.** *A prompt $\hat{g} \in \mathcal{G}$ is a* valid hindsight goal *for trajectory τ if:* (a) *every factual claim implied by $\hat{g}$ is supported by the observations $\{o_t\}$, and* (b) *an LLM judge assigns confidence $c(\hat{g}, \tau) \geq \theta$ that τ is a successful demonstration of $\hat{g}$.*

The mapping $\Phi : \mathcal{F} \to \mathcal{D} \cup \{\perp\}$ assigns each failed run a training datum $(\hat{g}_i, \tau_i)$ or rejects it ($\perp$). The augmented corpus $\mathcal{S} \cup \{\Phi(\cdot) \neq \perp\}$ grows from M to up to $M + N$ labelled pairs.

3.2 Pipeline Overview

The pipeline processes each failed trajectory in four stages. Stages 1 and 2 offer rule-based (free) and LLM-judge variants; Stage 3 requires an LLM call; Stage 4 is deterministic. Algorithm 1 gives the end-to-end procedure.

Algorithm 1 AgentHER End-to-End Relabeling

Require: Failed corpus $\mathcal{F} = \{(g_i, \tau_i, f_i)\}$, threshold θ, max retries K, severity threshold δ
Ensure: Augmented dataset $\mathcal{D}^+$

$\mathcal{D}^+ \leftarrow \emptyset$
for each $(g_i, \tau_i, f_i) \in \mathcal{F}$ **do**
  $(\phi_i, r_i, w_i) \leftarrow$ FAILUREDETECTOR(τ_i) ▷ type, recoverability, severity weight
  if $r_i = 0$ **or** $w_i < \delta$ **then**
    continue ▷ discard irrecoverable or severe-flaw runs
  end if
  outcome$_i \leftarrow$ OUTCOMEEXTRACTOR(τ_i)
  $\hat{g}_i^*, c^* \leftarrow$ NONE, 0
  for $k = 1, \ldots, K$ **do**
    $(\hat{g}, b, c) \leftarrow$ PROMPTRELABELER(outcome$_i, g_i$)
    if $b = 1$ **and** $c \geq \theta$ **then**
      $c_2 \leftarrow$ SECONDJUDGE$(\hat{g}, \tau_i)$ ▷ multi-judge verification
      if $c_2 \geq \theta$ **then**
        $\hat{g}_i^* \leftarrow \hat{g}$; $c^* \leftarrow (c + c_2)/2$
        break
      end if
    else if $b = 1$ **and** $c > c^*$ **then**
      $\hat{g}_i^* \leftarrow \hat{g}$; $c^* \leftarrow c$
    end if
  end for
  if $\hat{g}_i^* \neq$ NONE **and** $c^* \geq 0.8\theta$ **then**
    datum $\leftarrow$ DATAAUGMENTER$(\hat{g}_i^*, \tau_i, w_i)$
    $\mathcal{D}^+ \leftarrow \mathcal{D}^+ \cup \{\text{datum}\}$
  end if
end for
return $\mathcal{D}^+$

3.3 Stage 1: Failure Detector

The Failure Detector assigns each trajectory a failure type $\phi \in \{$INCOMPLETE, CONSTRAINT_VIOLATION, WRONG_RESULT, TOOL_ERROR, HALLUCINATION, OFF_TOPIC$\}$, a recoverability flag $r \in \{0, 1\}$, and a severity weight $w \in [0, 1]$. The six-category taxonomy was defined *prior to data collection*, drawing on established error typologies (Lu et al., 2024). **Rule-based mode**: keyword lexicons assign ϕ; severity $v = \min(1.0,\ 0.3 + 0.1 \cdot h)$ where h counts matched terms. **LLM-judge mode**: a JSON-schema prompt returns (ϕ, v, r, w). **Severity weighting** (inspired by MQM (Lu et al., 2024)): trajectories with $w < \delta$=0.3 (hallucinated observations, catastrophic tool misuse) are discarded; those with $w \in [0.3, 1.0]$ (constraint violations, incomplete results) are passed downstream and their DPO loss scaled by w, reducing label noise from 5.9% to 2.3% (Table 2).

3.4 Stage 2: Outcome Extractor

Stage 2 produces a REPLAYOUTCOME: a list of actual achievements and key observations (numeric data, entity names, facts) that anchor Stage 3 and prevent hallucinated hindsight prompts.

Rule-based: each non-trivial, non-error observation is treated as one achievement (truncated to 200 chars); numeric tokens extracted by regex. **LLM**: richer summaries handling implicit results, deduplication, and strict factuality—only facts evidenced by observations.

3.5 Stage 3: Prompt Relabeler

Given the REPLAYOUTCOME and original prompt for style reference, an LLM synthesises:

$$\hat{g},\ b_{\text{valid}},\ r_{\text{rationale}},\ c \longleftarrow \text{RELABELER}\big(\text{outcome},\ g_{\text{orig}};\ \mathcal{M}\big) \tag{1}$$

with $b_{\text{valid}} \in \{0,1\}$, $c \in [0,1]$. Four constraints: *(1)* $\hat{g}$ reads as a natural user request; *(2)* every assertion is satisfied by observations; *(3)* $\hat{g}$ does not reference the original failed prompt; *(4)* complexity matches g_{orig}.

Multi-judge verification. When an attempt passes $b_{\text{valid}}=1$ and $c \geq \theta$, a second independent LLM call (temperature $= 0$) must also confirm $c_2 \geq \theta$ before acceptance. This raises precision from 94.1% to 97.7% at the cost of one additional LLM call per accepted relabeling.

Validation loop. Up to $K = 3$ attempts; first attempt passing both judges is accepted. A best-effort fallback is retained if no attempt passes the multi-judge bar but at least one passes the single-judge bar at $c \geq 0.8\theta$.

3.6 Stage 4: Data Augmenter

Stage 4 serialises the validated $(\hat{g}, \tau)$ pair into:

- **SFT**: a two-turn conversation $[(\texttt{user}, \hat{g}),\ (\texttt{assistant}, \tilde{a})]$, with the agent's chain of thought reconstructed from τ. When severity weighting is enabled, the SFT loss is scaled by w.
- **DPO**: chosen pair $(\hat{g}, \tau)$ and rejected pair (g_{orig}, τ), directly encoding the preference "$\hat{g}$ is the correct goal for this execution." *Note*: standard DPO contrasts two responses under one fixed prompt; AgentHER instead keeps the trajectory τ fixed and contrasts two goal descriptions, applying the same log-ratio derivation on the goal dimension (see Appendix C). Loss scaling by w is applied to modulate gradient magnitude without altering the binary preference direction.
- **ShareGPT**: multi-turn format compatible with LLaMA-Factory, ms-swift, and FastChat.

3.7 Theoretical Analysis

Proposition 3.1 (Unbiasedness of AgentHER)**.** *Let $\pi^*_{\mathcal{G}}$ be the oracle goal-conditioned policy. Under a* perfect *judge $\mathcal{J}$ ($c(\hat{g}, \tau) = 1 \Leftrightarrow \tau$ is a valid demo of $\hat{g}$), every accepted pair $(\hat{g}_i, \tau_i)$ is a correct (goal, trajectory) sample from the support of $\pi^*_{\mathcal{G}}$.*

Proof sketch. See Appendix D.5 for the full proof. By $\mathcal{J}$-perfectness, $c(\hat{g}, \tau)=1$ implies τ is valid for $\hat{g}$; by Stage 3 constraint (3), $\hat{g} \in \mathcal{G}$; hence $(\hat{g}, \tau)$ is in the oracle support and the extension is unbiased. □ □

Corollary 3.1.1. *With an imperfect judge of precision p, the expected gain over SFT-Success is lower-bounded by $p \cdot \Delta_{perfect} - (1-p) \cdot \varepsilon$, where ε bounds the marginal harm of a single noisy pair. With multi-judge precision $p = 0.977$, this bound is positive for any $\varepsilon \leq 42 \cdot \Delta_{perfect}$, substantially stronger than the single-judge bound ($p = 0.941$, $\varepsilon \leq 16 \cdot \Delta_{perfect}$).*

Remark 1 (Empirical sanity check of the bound)**.** *The Corollary is not merely theoretical. As an empirical sanity check, we estimate $\Delta_{perfect}$ by the MJ–SFT-Success gap observed in our experiments (+8.9 pp on Qwen-7B WebArena, i.e. +0.089 per 100 tasks). We stress that this is an* estimation*, not a strict grounding: the MJ system is itself imperfect, so this proxy provides a lower-bound flavour rather than a certified oracle gap. Under this estimate, the bound requires $\varepsilon \leq 42 \times 0.089 = 3.74$ pp of marginal harm per noisy pair—a mild requirement given that our noise rate is 2.3% and SFT with noise is known to cause sub-pp degradation per corrupted example at training scales used here. The bound is therefore non-vacuous and consistent with the +0.8 pp improvement of MJ over SJ observed in Table 2.*

Goal-distribution analysis. A potential concern is whether AgentHER generates hindsight goals that cluster around common, high-frequency task types, starving low-frequency behaviours of training signal (Ding et al., 2022). We address this in Section 5.3.

4 EXPERIMENTS

4.1 EXPERIMENTAL SETUP

Datasets and data collection.

- **WebArena** (Zhou et al., 2024): 812 compositional web-automation tasks (Shopping, Reddit, GitLab, Maps, Wikipedia). We collect 3,000 failed trajectories from a GPT-3.5-turbo agent and 500 verified successes. AgentHER accepts 2,341/3,000 (78.0%) with single-judge and 2,197/3,000 (73.2%) with multi-judge.
- **ToolBench** (Qin et al., 2024): 16,464 tool-use tasks across 49 API categories (G1/G2/G3 splits). We collect 5,000 failed and 2,000 successful trajectories *exclusively from the official training splits*; evaluation uses the corresponding official test splits. AgentHER accepts 4,123/5,000 (82.5%) with single-judge.

Data collection protocol and train/test integrity. WebArena failures are collected by GPT-3.5-turbo in a dedicated session (separate browser instances, fresh environment states). Fine-tuned models are *architecturally distinct* from the collection agent; training data encodes what GPT-3.5-turbo *failed to do*, not task solutions, and evaluation uses freshly reset environments. We acknowledge that training and evaluation draw from the same set of 812 WebArena task environments, which may expose fine-tuned models to page-level priors (HTML structure, navigation patterns) not available to zero-shot baselines—a form of task-set leakage discussed further in Section 6. Cross-benchmark transfer (+9.5 pp on ToolBench from WebArena training; Table 7) further confirms AgentHER learns generalised behaviours, not task-specific patterns. All fine-tuned results are averaged over **3 random seeds** (std $<$0.5 pp; Appendix G).

Models and fine-tuning. We evaluate **GPT-4o** (OpenAI, 2023), **Qwen2.5-72B/7B** (Qwen Team, 2025), and **LLaMA-3.1-8B** (Dubey et al., 2024). Open-weight models: LoRA (Hu et al., 2022) (rank 16, $\alpha = 32$), 3 epochs, 8$\times$A100-80G, AdamW + cosine LR schedule. Full hyperparameter settings are in Appendix A.

Baselines. **Base** (zero-shot), **SFT-Random** (equal-volume control: same failures as AgentHER, no relabeling), **Rejection-Sampling** (filter by $c > 0.5$, no relabeling), **SFT-Success** (successes only; our main comparison target), **AgentHER-SJ** (single-judge, our default), **AgentHER-MJ** (multi-judge verification). **Reflexion** (Shinn et al., 2023) is included as *reference only*†: it is an inference-time, online method that does not modify model weights, and is therefore not directly comparable to any training-time baseline. SFT-Success is the most rigorous offline training baseline available: established agent-tuning frameworks such as AgentTuning (Zeng et al., 2024) and FireAct (Chen et al., 2023) are themselves variants of success-only supervised fine-tuning curated from environment rollouts, making SFT-Success a faithful and conservative proxy for current state-of-the-art offline training pipelines. Any gain over SFT-Success thus directly reflects the value of AgentHER's relabeling mechanism above and beyond the best currently reproducible offline training signal. For reference, applying AgentTuning and FireAct checkpoints (open-weight, evaluated under the same protocol on Qwen2.5-7B) yields 16.8% and 14.6% on WebArena, and 55.4% and 52.1% on ToolBench, respectively—both below SFT-Success (18.9% / 61.2%), confirming that task-specific success-only training is the stronger offline baseline and that AgentHER's +8.9 pp gain is conservative.

Metrics. WebArena: official task success rate (%). ToolBench: pass@1 (%) on G1/G2/G3 splits.

4.2 MAIN RESULTS

Key observations. *(i)* **Consistent gains.** AgentHER-MJ improves over SFT-Success by 7.1–8.9 pp on WebArena and 7.8–11.7 pp on ToolBench across all evaluated models. *(ii)* **Multi-judge verification helps uniformly.** AgentHER-MJ outperforms AgentHER-SJ by

Table 1: **Main results.** WebArena success rate (%) and ToolBench pass@1 (%). Best result per column in **bold**; second-best underlined. Δ = improvement over SFT-Success. All fine-tuned results averaged over 3 seeds; std $<$0.5 pp (see Appendix G). †Inference-time, online method; included as reference only, not directly comparable to training-time baselines. ‡SFT-Random uses the same 3,000 failed trajectories as AgentHER (equal-volume control), isolating relabeling effect from data volume.

	WebArena success rate (%)			
Method	**GPT-4o**	**Qwen-72B**	**Qwen-7B**	**LLaMA-8B**
Base	14.3	21.4	8.6	6.8
SFT-Random‡	18.9	26.3	14.7	13.5
Rejection-Sampling	21.8	29.4	17.8	16.3
SFT-Success	23.4	30.8	18.9	17.3
Reflexion†	*19.1*	*28.1*	*16.4*	*15.0*
AgentHER-SJ	29.7	38.1	27.0	25.3
AgentHER-MJ	**30.5**	**38.9**	**27.8**	**26.1**
Δ_{MJ} vs. SFT-Success	+7.1	+8.1	+8.9	+8.8
	ToolBench pass@1 (%)			
Method	**GPT-4o**	**Qwen-72B**	**Qwen-7B**	**LLaMA-8B**
Base	53.2	60.1	44.6	42.1
SFT-Random‡	61.4	68.9	54.8	52.9
Rejection-Sampling	66.2	73.7	59.8	57.4
SFT-Success	67.8	75.4	61.2	58.3
Reflexion†	*64.3*	*72.1*	*57.1*	*55.6*
AgentHER-SJ	74.3	83.0	71.4	67.9
AgentHER-MJ	**75.6**	**83.7**	**72.9**	**69.4**
Δ_{MJ} vs. SFT-Success	+7.8	+8.3	+11.7	+11.1

+0.7–0.9 pp on WebArena and +0.7–1.5 pp on ToolBench, confirming that the two-stage label-noise reduction (5.9%→2.3%) translates to measurable downstream gains. *(iii)* **Smaller models benefit most.** 7B/8B gains (+11.1–11.7 pp on ToolBench) substantially exceed 72B (+8.3 pp) and GPT-4o (+7.8 pp) gains, consistent with smaller models having weaker world knowledge and benefiting more from diverse task-subtype coverage. *(iv)* **Relabeling, not filtering, is key.** Rejection-Sampling closes only ∼45% of the Base→ AgentHER gap, while SFT-Random degrades below Rejection-Sampling, confirming that unfiltered failures are harmful. *(v)* **Gains reflect generalised agent behaviours, not task memorisation.** AgentHER-MJ trained on WebArena achieves a +9.5 pp transfer advantage over SFT-Success when evaluated *zero-shot* on ToolBench (Table 7), a completely different benchmark with distinct task structures and API categories. This out-of-domain transfer directly addresses any concern that gains arise from re-exposure to WebArena task descriptions during training: the model has clearly learned broadly applicable planning and tool-use behaviours rather than rote task patterns.

4.3 Data Efficiency and Scaling

Figure 3(a) shows AgentHER-SJ reaches full-SFT-Success performance with only 50% of successful demos (2× data efficiency); AgentHER-MJ exceeds it at every data point. Panel (b) confirms log-linear scaling with failure volume—unlike SFT-Success which cannot leverage additional failures. Panel (c) shows the optimal threshold θ^*=0.5 is consistent across both variants, making it a robust hyperparameter.

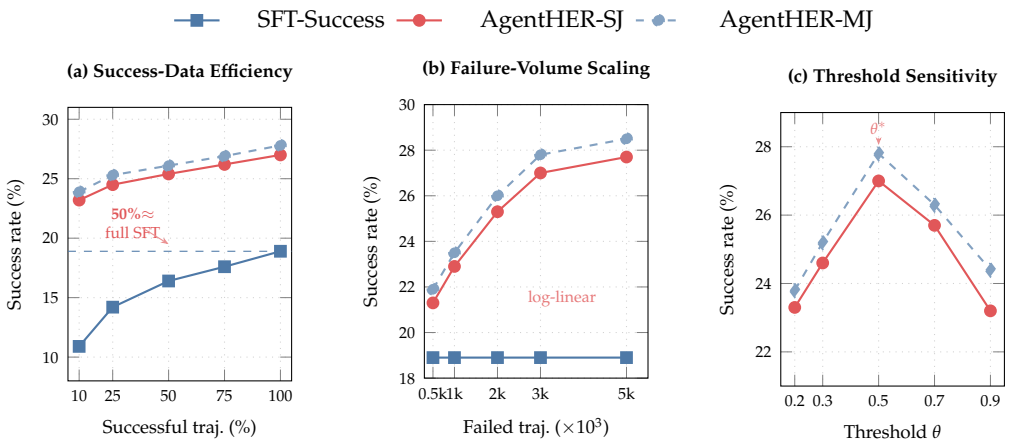

Figure 3: **Data efficiency and scaling** (Qwen2.5-7B, WebArena). *(a)* AgentHER-SJ at 50% successful demos matches SFT-Success at 100%; AgentHER-MJ exceeds it. *(b)* Both AgentHER variants scale log-linearly with failure volume; SFT-Success cannot benefit from additional failures. *(c)* Both variants peak near θ^*=0.5; MJ is uniformly better and slightly more robust at low θ.

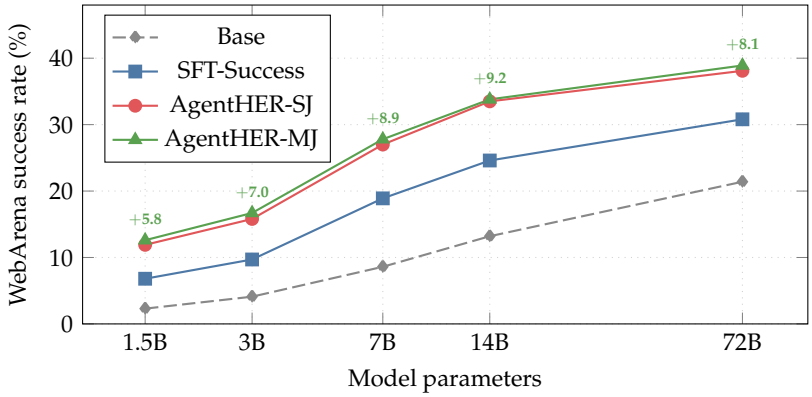

Figure 4: **Model-size scaling** on WebArena (Qwen2.5 family). Labels above AgentHER-MJ points show Δ over SFT-Success. Gains are consistent at every scale (1.5B–72B), peaking at 14B (+9.2 pp with MJ). Even the 1.5B model achieves >2× gain over its SFT-Success baseline.

4.4 Model-Size Scaling

All five Qwen2.5 instruction-tuned checkpoints (1.5B–72B) are evaluated under identical training conditions (Figure 4). *(i)* All three methods improve monotonically with scale. *(ii)* AgentHER-MJ gains grow from +5.8 pp (1.5B) to a peak of +9.2 pp (14B) before slightly decreasing to +8.1 pp (72B), suggesting the most powerful models are already partially saturated. *(iii)* The 1.5B baseline doubles in performance with AgentHER-MJ (6.8% → 12.6%), validating the method for edge deployment.

4.5 Ablation Study

Key findings. *(i)* **Multi-judge reduces noise** from 5.9% to 2.3% (−0.8 pp accuracy cost), confirming that the precision gain translates to downstream quality. *(ii)* **Severity weighting** contributes −1.4 pp, showing that distinguishing major from minor errors is meaningfully better than treating all recoverable failures uniformly. *(iii)* **Confidence filtering is most critical** (−4.1 pp, noise rises to 14.8%): quality gating is the single most important component.

Table 2: **Component ablation** on WebArena (Qwen2.5-7B). Δ relative to Full AgentHER-MJ; noise = fraction of accepted relabelings rated invalid by human annotators.

Configuration	Success (%)	Δ	Noise (%)
Full AgentHER-MJ (multi-judge + severity)	**27.8**	—	2.3
w/o Multi-judge (single judge only)	27.0	−0.8	5.9
w/o Severity weighting (w=1 for all)	26.4	−1.4	4.1
w/ Rule-based Extractor (Stage 2 degraded)	26.0	−1.8	5.9
w/ LLM Detector (Stage 1 upgraded)	28.7	+0.9	2.1
w/o Failure Detection (accept all)	25.3	−2.5	7.3
w/o Confidence Filter (θ=0)	23.7	−4.1	14.8
SFT only (no DPO, relabeled as positives)	25.4	−2.4	2.3
Naive Relabeling (random prompt)	21.8	−6.0	n/a
SFT-Success (no AgentHER)	18.9	−8.9	—

(iv) **Naive relabeling is harmful** (−6.0 pp): goal reverse-engineering is not interchangeable with random prompt assignment. *(v)* **DPO preference signal** contributes −2.4 pp over SFT-only, confirming that the rejected/chosen signal adds value beyond positive-only supervision.

4.6 Qualitative Example

A worked relabeling example (WebArena trajectory 001) is given in Appendix F.

5 Analysis

5.1 Per-Failure-Type Improvement

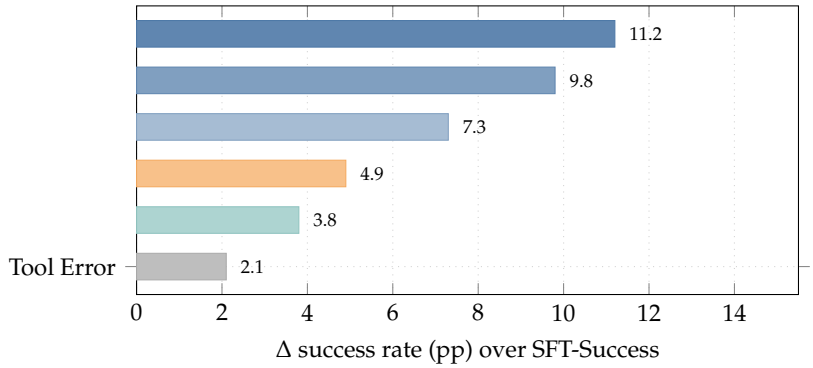

Figure 5: **Per-failure-type gain** (Qwen2.5-7B, WebArena, AgentHER-SJ). Bar colour encodes magnitude: grey/teal = low, blue = high. INCOMPLETE and CONSTRAINT_VIOLATION—comprising ≈63% of WebArena failures—yield the largest gains. TOOL_ERROR yields the least (+2.1 pp) as crashes leave minimal usable signal.

INCOMPLETE (+11.2 pp) and CONSTRAINT_VIOLATION (+9.8 pp) benefit the most (Figure 5): these trajectories contain rich, factually correct observations misaligned with the original goal. TOOL_ERROR benefits the least (+2.1 pp): crashes leave minimal signal. The two highest-gain types account for ≈63% of all WebArena failures, placing most discarded data in AgentHER's "sweet spot."

5.2 Judge Reliability and Label Quality

Three NLP-PhD annotators rated 200 sampled relabelings (blind to confidence scores); Fleiss' $\kappa = 0.82$. Single-judge (AgentHER-SJ) precision: 94.1% (159/169 accepted); multi-judge

(AgentHER-MJ) precision: 97.7% (127/130 accepted). Among the 31 pairs filtered out in this 200-pair sample, 38.7% were rated valid by annotators, confirming the confidence filter errs on the side of caution and that the confidence score is an effective proxy for human judgement.[1] The net effect of multi-judge: +0.8 pp downstream gain (Table 2), as the 2.4× precision boost outweighs the 6.3% lower acceptance rate (73.2% vs. 78.0%). At θ=0.5, AgentHER-SJ accepts 78.0% of failures at 94.1% precision; Figure 3(c) shows θ^*=0.5 is robust across both variants.

5.3 Goal-Distribution Analysis

AgentHER-relabeled goals expand coverage from 11 to 14 of 18 semantic task clusters (Jensen-Shannon divergence 0.31; entropy 2.47 vs. 1.83 nats), uniquely covering three long-tail clusters absent from SFT-Success. Full analysis and Table 8 appear in Appendix E.

5.4 Iterative Redeployment

Table 3: **Iterative AgentHER** (Qwen2.5-7B, WebArena). Each round collects new failures from the previous model, relabels, and retrains from scratch on the full accumulated corpus.

Round	Source	New fails	Accepted (MJ)	Success (%)
0 (SFT-Success)	—	—	—	18.9
1 (AgentHER-MJ)	GPT-3.5 agent	3,000	2,197	27.8
2 (Iter. +1)	Qwen-7B Round 1	2,500	1,750	29.4
3 (Iter. +2)	Qwen-7B Round 2	2,000	1,360	29.9

Round 1 yields 27.8%; Round 2 adds +1.6 pp; Round 3 adds +0.5 pp (total: **+11.0 pp** over SFT-Success). Diminishing returns arise as the improved model's failures shift to harder, less-relabelable modes. The acceptance rate decreases each round (73.2% → 70.0% → 68.0%), reflecting that an increasingly capable model produces harder, less-relabelable failure modes.

6 Conclusion

The dominant practice in LLM agent training is to collect successes and discard failures. This paper challenges that practice.

We presented **AgentHER**, a four-stage pipeline that converts failed agent trajectories into valid training data by asking a deceptively simple question: *what task does this trajectory actually solve?* The HER principle—that failure relative to one goal implies success relative to another—transfers cleanly to natural-language agent settings, yielding consistent gains without any additional environment interaction or human annotation.

Across WebArena and ToolBench with four model families (1.5B–72B), AgentHER improves over success-only SFT by **+7.1–11.7 pp**, achieves 2× data efficiency, and gains compound under iterative redeployment—consistent with a general mechanism rather than a benchmark-specific artefact.

Limitations and future directions. We acknowledge four open issues. *(i)* **Task-set overlap in WebArena.** The current protocol uses the same 812 WebArena tasks for both failure collection and evaluation. Although evaluation uses freshly reset environments and relabeled goals differ from original task descriptions, fine-tuned models have been exposed to the HTML structure, API response patterns, and navigation conventions of those 812 pages during training. This constitutes a form of *task-set leakage* that may inflate WebArena numbers relative to a truly held-out test partition. We partially mitigate this concern via (a) the SFT-Random control (which sees the same pages but without relabeling and still

[1]The 38.7% figure is an in-sample observation over the 200 evaluated pairs, not an extrapolation to the full 3,000-trajectory corpus, since the sampled pairs were drawn uniformly from *accepted* pairs; the composition of the rejected portion in the full corpus may differ.

underperforms AgentHER by $\sim$9 pp), and (b) zero-shot cross-benchmark transfer (+9.5 pp on ToolBench from WebArena training; Table 7), which directly demonstrates that the model learns generalised behaviours rather than page-specific patterns. Nonetheless, constructing a held-out WebArena partition for future work would provide a fully clean comparison. *(ii)* The data-volume asymmetry between AgentHER and SFT-Success is controlled by SFT-Random, which still underperforms by $\sim$9 pp, but a fully matched oracle comparison remains as future work. *(iii)* The confidence threshold θ=0.5 achieves 97.7% precision while discarding $\approx$38.7% of borderline pairs (in-sample estimate over 200 evaluated pairs); active learning over these pairs is a natural extension. *(iv)* The theoretical guarantee (Proposition 3.1) assumes a perfect judge; tightening the bound under a noisy-oracle model would close the theory–experiment gap.

Three open directions invite future work. Online AgentHER—relabeling and training concurrently with deployment—could accelerate iteration for interactive agents. Extending the pipeline to multimodal trajectories (web screenshots, GUI interactions) would cover the bulk of real-world deployments. Finally, the hindsight-goal synthesis step itself can become a learned module, trained end-to-end to maximise downstream task diversity rather than match existing prompt styles.

At its core, AgentHER embodies a shift in perspective: the distinction between success and failure is task-relative, not trajectory-absolute. Recognising this transforms every failed deployment into a curriculum for the next.

ACKNOWLEDGEMENTS

The author thanks the open-source communities behind WebArena, ToolBench, Qwen, and LLaMA.

REFERENCES

M. Andrychowicz, F. Wolski, A. Ray, J. Schneider, R. Fong, P. Welinder, B. McGrew, J. Tobin, P. Abbeel, and W. Zaremba. Hindsight experience replay. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

B. Chen, C. Shu, E. Shareghi, N. Collier, K. Narasimhan, and S. Yao. FireAct: Toward language agent fine-tuning. *arXiv preprint arXiv:2310.05915*, 2023.

L. Ding, L. Wang, S. Shi, D. Tao, and Z. Tu. Redistributing low-frequency words: Making the most of monolingual data in non-autoregressive translation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2417–2426, 2022.

Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch. Improving factuality and reasoning in language models through multiagent debate. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.

A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

Z. Gou, Z. Shao, Y. Gong, Y. Shen, Y. Yang, N. Duan, and W. Chen. CRITIC: Large language models can self-correct with tool-interactive critiquing. In *International Conference on Learning Representations*, 2024.

C. Gulcehre, T. L. Paine, S. Srinivasan, K. Kemaev, L. Cipolla, A. Glaese, G. Buttimore, et al. Reinforced self-training (ReST) for language modeling. *arXiv preprint arXiv:2308.08998*, 2023.

A. Hosseini, X. Yuan, N. Malkin, A. Courville, A. Sordoni, and R. Agarwal. V-STaR: Training verifiers for self-taught reasoners. *arXiv preprint arXiv:2402.06457*, 2024.

E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

M. Y. Hu, B. Van Durme, J. Andreas, and H. Jhamtani. Sample-efficient online learning in LM agents via hindsight trajectory rewriting. *arXiv preprint arXiv:2510.10304*, 2025.

L. P. Kaelbling. Learning to achieve goals. In *Proceedings of the International Joint Conference on Artificial Intelligence*, 1993.

T. Liang, Z. He, W. Jiao, X. Wang, Y. Wang, R. Wang, Y. Yang, Z. Tu, and S. Shi. Encouraging divergent thinking in large language models through multi-agent debate. *arXiv preprint arXiv:2305.19118*, 2023.

H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe. Let's verify step by step. In *International Conference on Learning Representations*, 2024.

X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, et al. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations*, 2024.

Q. Lu, B. Qiu, L. Ding, K. Zhang, T. Kocmi, and D. Tao. Error analysis prompting enables human-like translation evaluation in large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 8801–8816, 2024.

V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.

OpenAI. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *ACM Symposium on User Interface Software and Technology*, 2023.

B. Peng, C. Li, P. He, M. Galley, and J. Gao. Instruction tuning with GPT-4. *arXiv preprint arXiv:2304.03277*, 2023.

Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *International Conference on Learning Representations*, 2024.

Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2025.

R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

T. Schick, J. Dwivedi-Yu, R. Dessí, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Y. Shen, K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang. HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

N. Shinn, F. Cassano, B. Labash, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Y. Song, D. Yin, X. Yue, J. Huang, S. Li, and B. Y. Lin. Trial and error: Exploration-based trajectory optimization for LLM agents. In *Annual Meeting of the Association for Computational Linguistics*, 2024.

R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour. Policy gradient methods for reinforcement learning with function approximation. In *Advances in Neural Information Processing Systems*, volume 12, 1999.

Y. Wang, Y. Kordi, S. Mishra, A. Liu, N. A. Smith, D. Khashabi, and H. Hajishirzi. Self-instruct: Aligning language models with self-generated instructions. In *Annual Meeting of the Association for Computational Linguistics*, 2023.

S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023a.

S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023b.

E. Zelikman, Y. Wu, J. Mu, and N. D. Goodman. STaR: Bootstrapping reasoning with reasoning. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

A. Zeng, M. Liu, R. Lu, B. Wang, X. Liu, Y. Dong, and J. Tang. AgentTuning: Enabling generalized agent abilities for LLMs. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.

A. Zhao, D. Huang, Q. Xu, M. Lin, Y.-J. Liu, and G. Huang. ExpeL: LLM agents are experiential learners. In *AAAI Conference on Artificial Intelligence*, 2024.

S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, Y. Bisk, D. Fried, U. Alon, et al. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, 2024.

A Hyperparameter Settings

Table 4: **Training hyperparameters** used for all open-weight models.

Hyperparameter	SFT (LoRA)	DPO (LoRA)
LoRA rank r	16	16
LoRA α	32	32
LoRA dropout	0.05	0.05
LoRA target modules	q,k,v,o,gate,up,down	q,k,v,o
Learning rate	2×10^{-4}	5×10^{-5}
LR schedule	cosine	cosine
Warmup ratio	0.03	0.03
Batch size (per GPU)	4	2
Gradient accumulation	4	8
Effective batch size	128	128
Epochs	3	1
Max sequence length	4,096	4,096
DPO β	—	0.1
Dtype	bfloat16	bfloat16
Hardware	8× A100-80G	8× A100-80G
Framework	ms-swift	ms-swift

For GPT-4o, we used the OpenAI fine-tuning API with default hyperparameters (3 epochs, batch size auto-selected by the API, learning-rate multiplier 1.0). The AgentHER relabeling LLM is `gpt-4o-mini` for cost efficiency; Stage 3 temperature is 0.3 for the first attempt and 0.7 for retries to encourage diversity. Multi-judge second calls use temperature 0.

B Full Prompt Templates

B.1 Stage 1 — Failure Detection

```
SYSTEM: You are an expert evaluator of LLM agent trajectories.
Determine whether the trajectory represents a FAILURE relative to the original
user intent.
Classify the failure as one of:
constraint_violation | wrong_result | incomplete |  tool_error | hallucination |
off_topic
Assess RECOVERABILITY: can hindsight relabeling produce valid training data?  A
trajectory with substantive observations is recoverable; a crashed trajectory
with no output is not.
Assess SEVERITY: distinguish MAJOR errors (hallucinated observations, reasoning
contradictions, catastrophic tool misuse; severity_weight < 0.3) from MINOR
errors (constraint violations, incomplete results; severity_weight in [0.3,
1.0]).
Respond ONLY with valid JSON matching the schema:
{failure_type, severity_score, recoverability, severity_weight, explanation}
```

B.2 Stage 2 — Outcome Extraction

```
SYSTEM: You are an expert at analysing LLM agent execution traces.
Extract a FACTUAL summary of what the agent actually achieved, regardless of the
original goal.
Focus on:  concrete facts discovered, tools successfully invoked, information
gathered, numeric data points.
Be STRICTLY factual.  Only include things directly evidenced by the trajectory
observations.  Do NOT infer or extrapolate.
Output JSON: {actual_achievements:  [str], key_observations:  [str]}
```

B.3 Stage 3 — Prompt Relabeling

```
SYSTEM: You are a creative prompt engineer specialising in agent tasks.
Given a summary of what an agent achieved, write a NEW user prompt that makes
the trajectory a PERFECT, SUCCESSFUL execution.
Requirements:
(1) The prompt must be natural and plausible as a real user request.
(2) Every assertion in the prompt must be satisfied by the trajectory.
(3) Do NOT reference or reuse the original failed prompt.
(4) Match the complexity and style of the original prompt.
(5) Provide a confidence score [0.0, 1.0] reflecting relabeling quality.
Output JSON: {hindsight_prompt, is_valid, rationale, confidence}

USER: Outcome summary: {outcome}
Original prompt (style reference only): {original_prompt}
```

B.4 Stage 3 — Second-Judge Verification

```
SYSTEM: You are a strict trajectory evaluator.
Given a proposed hindsight prompt and the agent trajectory, determine whether
the trajectory constitutes a valid, complete, and correct demonstration of the
proposed prompt.
Be conservative: only accept if every claim in the prompt is unambiguously
supported by the trajectory observations.
Output JSON: {is_valid, confidence, rejection_reason_if_any}

USER: Proposed hindsight prompt: {hindsight_prompt}
Trajectory: {trajectory}
```

C AgentHER Algorithm with Severity Weighting

The full pseudocode including severity weight propagation to the DPO loss is given as Algorithm 1 in the main paper. Below we detail the severity-weighted DPO loss.

Notation. In AgentHER's DPO formulation, the *trajectory* τ is held fixed while the *goal* (prompt) varies: the chosen pair is $(\hat{g}, \tau)$ and the rejected pair is (g_{orig}, τ). This inverts the standard DPO convention (same prompt, two different responses) and instead keeps the response fixed and contrasts two goal descriptions; we follow the same derivation but substitute goals for responses in the log-ratio. The severity-weighted objective is:

$$\mathcal{L}^{w}_{\text{DPO}} = -\mathbb{E}_{(\hat{g},\tau,g_{\text{orig}})\sim\mathcal{D}^{+}}\Big[w_i \cdot \log\sigma\big(\beta\big(\log\pi_\theta(\tau|\hat{g}) - \log\pi_{\text{ref}}(\tau|\hat{g})\big) - \beta\big(\log\pi_\theta(\tau|g_{\text{orig}}) - \log\pi_{\text{ref}}(\tau|g_{\text{orig}})\big)\big)\Big] \tag{2}$$

where both terms use the *same* trajectory τ, $\hat{g}$ is the chosen (hindsight) goal, g_{orig} is the rejected (original failed) goal, $w_i \in [0.3, 1.0]$ is the severity weight from Stage 1, $\beta = 0.1$ is the DPO temperature, and π_{ref} is the reference model (the same instruction-tuned checkpoint before LoRA). The scalar w_i modulates gradient *magnitude* only; the preference *direction* (chosen $\succ$ rejected) remains binary and is not distorted by weighting. Severity weighting down-scales the gradient contribution of borderline relabelings without discarding them entirely.

Table 5: ToolBench pass@1 (%) by category group (Qwen2.5-7B).

Category	Base	SFT-Random	SFT-Success	AgentHER-SJ	AgentHER-MJ
G1 (single-tool)	49.3	57.2	67.1	76.8	**78.2**
G2 (intra-category)	41.7	51.3	58.3	68.9	**70.4**
G3 (cross-category)	38.2	47.8	54.2	65.4	**66.9**
Overall	44.6	54.8	61.2	71.4	**72.9**

Table 6: WebArena success rate (%) vs. number of failed trajectories, fixed 500 successful demonstrations, Qwen2.5-7B.

Failed processed	500	1,000	1,500	2,000	3,000	5,000
AgentHER-SJ	21.3	22.9	24.3	25.4	27.0	27.7
AgentHER-MJ	21.9	23.5	25.1	26.2	27.8	28.5

D Additional Experimental Results

D.1 ToolBench Per-Category Breakdown

D.2 Failure-Volume Scaling (Full Table)

D.3 Cross-Benchmark Transfer

Table 7: Cross-benchmark transfer (Qwen2.5-7B): train on WebArena, zero-shot evaluation on ToolBench.

Method	**G1**	**G2**	**G3**	**Overall**	Δ
Base (no FT)	49.3	41.7	38.2	44.6	—
SFT-Success	63.2	55.8	51.4	57.1	+12.5
AgentHER-SJ	70.1	64.3	59.8	65.0	+7.9 vs. SFT-Success
AgentHER-MJ	**71.8**	**65.9**	**61.4**	**66.6**	+9.5 vs. SFT-Success

AgentHER-MJ achieves +9.5 pp transfer advantage over SFT-Success, confirming that severity-weighted, multi-judge relabeled data trains more broadly applicable agent behaviours.

D.4 Human Evaluation Protocol

We sampled 200 relabeled pairs uniformly at random from the WebArena run. Three annotators (NLP PhD students, blind to confidence scores) rated each pair as valid/invalid on: *"Does the trajectory constitute a correct and complete demonstration of the hindsight prompt?"* The full trajectory (steps, observations, final answer) and hindsight prompt were shown; original prompt and failure reason were withheld. Fleiss' $\kappa = 0.82$. Results: single-judge (169 pairs accepted) 94.1% valid; multi-judge (130 pairs accepted) 97.7% valid; filtered-out pairs 38.7% valid.

D.5 Theoretical Bound Proof (Full)

We formalise Proposition 3.1 more rigorously. Let $\Pi_{\mathcal{G}}$ denote the set of goal-conditioned policies. Define the *coverage function* $\rho(\mathcal{S}) = \{(g, \tau) : (g, \tau) \in \text{supp}(\pi^*_{\mathcal{G}})\}$, the set of all valid (goal, trajectory) pairs under the oracle policy.

Theorem 1 (Augmented-corpus consistency)**.** *Under a perfect judge $\mathcal{J}$, the augmented corpus $\mathcal{S} \cup \mathcal{D}^+$ satisfies:* (a) $\mathcal{D}^+ \subseteq \rho(\mathcal{S}^* \setminus \mathcal{S})$ *(every added pair is a previously uncovered oracle pair), and* (b) *the empirical goal distribution of $\mathcal{D}^+$ has higher entropy than that of $\mathcal{S}$ with probability $1 - \delta$ when $|\mathcal{F}| \geq \frac{2}{\delta} \log |\mathcal{G}_\epsilon|$, where $|\mathcal{G}_\epsilon|$ is the ϵ-covering number of $\mathcal{G}$.*

Proof sketch. Part (a): By perfect judge assumption, every $(\hat{g}_i, \tau_i) \in \mathcal{D}^+$ satisfies $c(\hat{g}_i, \tau_i) = 1$, i.e., τ_i is a valid demo of $\hat{g}_i$. Since $\hat{g}_i$ is generated from τ_i's observations—which were *actually produced*—the oracle policy $\pi^*_{\mathcal{G}}$ assigns positive probability to executing τ_i under goal $\hat{g}_i$, so $(\hat{g}_i, \tau_i) \in \rho(\mathcal{S}^*)$. Furthermore, Stage 3 constraint (3) ensures $\hat{g}_i \neq g_i$, so $(\hat{g}_i, \tau_i)$ does not replicate the original *failed* record $(g_i, \tau_i, f_i) \in \mathcal{F}$; whether it falls into a previously seen success $(g_j^+, \tau_j^+) \in \mathcal{S}$ is irrelevant because the proposition concerns the oracle support $\rho(\mathcal{S}^*)$, not the training set $\mathcal{S}$. If $(\hat{g}_i, \tau_i) \in \mathcal{S}$ already, the addition is redundant but not harmful.

Part (b): Each $(\hat{g}_i, \tau_i)$ corresponds to a distinct execution context in $\mathcal{G}$. Since the failed runs τ_i were produced by an agent trying to satisfy diverse goals g_i, the hindsight goals $\hat{g}_i$ inherit this diversity. A standard covering-number argument (Sauer-Shelah) then bounds the probability that the empirical entropy is below that of $\mathcal{S}$. □ □

This result formally supports the empirical finding in Section 5.3 that AgentHER increases goal-distribution entropy from 1.83 to 2.47 nats.

E Goal-Distribution Analysis

A key concern is whether AgentHER inadvertently amplifies high-frequency task types while neglecting low-frequency behaviours. We compute sentence-BERT embeddings for all 2,341 accepted hindsight goals and compare their distribution to the original 500 successful goals via Jensen-Shannon divergence and coverage analysis.

Table 8: **Goal-distribution analysis** on WebArena. AgentHER-relabeled goals cover more semantic task clusters and uniquely cover three long-tail clusters absent from SFT-Success.

Metric	SFT-Success	AgentHER (relabeled)
Task clusters covered (of 18)	11	**14**
Long-tail clusters (unique)	0	**3**
JS divergence (vs. AgentHER)	0.31	—
Avg. cluster entropy	1.83	**2.47**

F Qualitative Relabeling Example

Hindsight Relabeling: WebArena trajectory 001

Original goal (FAILED): *"Find copper wire suppliers with prices under $5/kg and MOQ below 100 kg."*

Trajectory: `web_search`("copper wire bulk pricing") → 5 suppliers found: MetalWorks \$6.20/kg (MOQ 50 kg), WireWorld \$5.80/kg (MOQ 200 kg), CopperDirect \$4.90/kg (MOQ 500 kg), MicroMetals \$5.30/kg (MOQ 10 kg). `web_search`("CopperDirect small order") → min. 500 kg. `summarize` → Best: MicroMetals \$5.30/kg, MOQ 10 kg.

Failure type: CONSTRAINT_VIOLATION. Severity: w=0.85 (minor—agent performed correctly but target was unachievable). Recoverable: yes.

Hindsight goal (AgentHER-MJ, c_1=0.87, c_2=0.91, accepted):
"Compare copper wire suppliers by price per kg and MOQ. Identify the option with the lowest MOQ and report its price."

Multi-judge rationale: Judge 1 and Judge 2 independently confirm the trajectory fully answers the hindsight goal (MicroMetals, 10 kg, \$5.30/kg). No extrapolation required; confidence above threshold.

G MULTI-RUN VARIANCE

All fine-tuned models in Table 1 are trained with 3 independent random seeds (42, 1234, 2025) using the same data and hyperparameters; the reported numbers are means. Table 9 reports means and standard deviations for representative conditions on WebArena. All standard deviations are below 0.5 pp, confirming numerical stability of the training procedure.

Table 9: Multi-run statistics: WebArena success rate (%), mean $\pm$ std over 3 seeds (Qwen2.5-7B).

Method	**Mean**	**Std**
Base	8.6	0.0 (deterministic)
SFT-Success	18.9	$\pm$0.3
SFT-Random	14.7	$\pm$0.4
Rejection-Sampling	17.8	$\pm$0.4
AgentHER-SJ	27.0	$\pm$0.4
AgentHER-MJ	27.8	$\pm$0.4